INDEX GENERATION
Advanced Lab Exam Modification Questions

Practice guide based on the supplied Trie, BST, Dictionary and Benchmark project

Note: These are plausible practice questions grounded in the supplied project code and specification, not confirmed predictions of the exact exam paper.

How to use this guide

• Attempt each modification first, without looking at the answer.

• Identify the function responsible for the behavior before editing.

• Test on a tiny file containing repeated words, punctuation, a blank line and a missing search term.

• Keep Trie, BST and dictionary behavior consistent when a feature applies to all three.

Part A — Core project modifications

Q1. Replace input() with command-line arguments

Task: The specification requires input and output filenames to be passed on the command line.

```
import sys

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("Usage: python dict_index.py <input> <output>")
        raise SystemExit(1)
    infile = sys.argv[1]
    outfile = sys.argv[2]
    write(build(infile), outfile)
```

Apply the same main-block change in all three modules, retaining each module's own build() and write(). Test with: python dict_index.py sample.txt index.txt.

Q2. Make indexing case-insensitive

```
def get_words(line):
    for sep in ".,:;":
        line = line.replace(sep, " ")
    return line.lower().split()

# First line inside each search() function:
word = word.lower()
```

Build and search must normalize in the same way. Otherwise a stored 'Tree' and query 'tree' may not match. This intentionally changes the original case-sensitive behavior.

Q3. Add a new punctuation separator

```
def get_words(line):
    for sep in ".,:;?'":
        line = line.replace(sep, " ")
    return line.split()
```

Only the shared tokenizer changes. The original specification permits certain separators; this extends the behavior to question marks and apostrophes. Do not filter with isalpha(), because printable ASCII characters are allowed.

Q4. Count distinct words in the dictionary

```
index = build(infile)
print("Distinct words:", len(index))
write(index, outfile)
```

For BST, count nodes. For Trie, count only terminal nodes where node.lines is not None; intermediate character nodes are not necessarily complete words.

Q5. Use safer file handling with with open()

```
def build(filename):
    index = {}
    with open(filename, "r", encoding="utf-8") as f:
        for num, line in enumerate(f, start=1):
            for word in get_words(line):
                if word not in index:
                    index[word] = LineQueue()
                index[word].add(num)
    return index
```

This is the dictionary version. Trie and BST use the same context-manager pattern but different insertion statements. The file closes automatically, even if an error occurs.

Part B — Linked-list queue and indexing semantics

Q6. Suppress duplicate line numbers anywhere in the queue

```
def add(self, line):
    cur = self.head
    while cur is not None:
        if cur.line == line:
            return
        cur = cur.next

    node = Node(line)
    if self.head is None:
        self.head = node
    else:
        self.tail.next = node
    self.tail = node
```

The original optimization checks only the tail. A full scan handles calls such as add(2), add(5), add(2), but takes O(k) time per add for a queue of length k. Expected result: '2, 5'.

Q7. Return the queue's line numbers as a list

```
def to_list(self):
    result = []
    cur = self.head
    while cur is not None:
        result.append(cur.line)
        cur = cur.next
    return result
```

Time O(k), extra space O(k). This helper is useful for tests and comparison with expected output.

Q8. Count queue nodes

```
def count(self):
    total = 0
    cur = self.head
    while cur is not None:
        total += 1
        cur = cur.next
    return total
```

Under original queue semantics, this counts distinct indexed lines for a word, not total occurrences.

Q9. Track total frequency separately from line numbers

```
# Dictionary build loop:
if word not in index:
    index[word] = {"lines": LineQueue(), "frequency": 0}
index[word]["frequency"] += 1
index[word]["lines"].add(num)

# Dictionary output:
for word in sorted(index):
    item = index[word]
    out.write(
        f"{word}: {item['lines'].text()} "
        f"(frequency={item['frequency']})\n"
    )
```

This changes the dictionary value type, so update search() and every other place that expects index[word] to be a LineQueue. Frequency means total occurrences; line list means where the word occurs.

Q10. Ignore blank lines without renumbering later lines

```
for num, line in enumerate(f, start=1):
    if not line.strip():
        continue
    for word in get_words(line):
        # original insertion logic
        ...
```

enumerate() counts physical lines, including blank ones. Skipping tokenization for a blank line must not cause later line numbers to shift.

Part C — Search and output

Q11. Handle a missing word safely

```
queue = search(index, query)
if queue is None:
    print("Word not found")
else:
```

```
    print(query + ": " + queue.text())
```

Never call .text() on None. This pattern works for the three implementations because search() returns a queue or None.

Q12. Add a dictionary contains() helper

```
def contains(index, word):
    return word in index
```

For Trie/BST, use return search(root, word) is not None. Keep search() if existing code relies on its queue result.

Q13. Output only words found on at least N lines

```
def write(index, outname, min_lines=1):
    with open(outname, "w", encoding="utf-8") as out:
        for word in sorted(index):
            if index[word].count() >= min_lines:
                out.write(word + ": " + index[word].text() + "\n")
```

Requires Q8's count() method. For Trie, apply the condition at terminal nodes; for BST, before writing each node. This counts lines, not occurrences.

Q14. Add Trie prefix search

```
def prefix_search(root, prefix):
    node = root
    for ch in prefix:
        if ch not in node.children:
            return []
        node = node.children[ch]

    found = []
    def collect(cur, word):
        if cur.lines is not None:
            found.append(word)
        for ch in sorted(cur.children):
            collect(cur.children[ch], word + ch)

    collect(node, prefix)
    return found
```

First follow the prefix path. Then collect complete words in that subtree. The lines check matters because a prefix node is not always a complete word.

Part D — BST and Trie structure changes

Q15. Count distinct words in the BST

```
def count_words(node):
    if node is None:
        return 0
    return (1 + count_words(node.left)
              + count_words(node.right))
```

Each BST node represents one distinct word. Time O(n), recursion stack O(h), where n is node count and h is tree height.

Q16. Print BST output iteratively

```
def print_tree(root, out):
    stack = []
    node = root
    while stack or node is not None:
        while node is not None:
            stack.append(node)
            node = node.left
        node = stack.pop()
        out.write(node.word + ": " + node.lines.text() + "\n")
        node = node.right
```

This is iterative in-order traversal and preserves lexicographic order.

Q17. Count complete words in a Trie

```
def count_words(node):
    total = 1 if node.lines is not None else 0
    for child in node.children.values():
        total += count_words(child)
    return total
```

Only terminal nodes (node.lines is not None) represent complete indexed words.

Q18. Explain a BST worst case

If words arrive in sorted order, the ordinary unbalanced BST may become a chain. Search/insertion can degrade to O(n), whereas a reasonably balanced tree typically takes O(log n). The supplied BST is not self-balancing.

Part E — Benchmark modifications

Q19. Make random benchmark data reproducible

```
random.seed(42)
```

Place this after importing random and before generating files or queries. The same seed makes the pseudo-random sequence repeatable for the same code path.

Q20. Report average search time per query

```
query_count = len(queries)
if query_count:
    avg_seconds = t_search / query_count
    avg_microseconds = avg_seconds * 1_000_000
    print("Average search (microseconds):",
          round(avg_microseconds, 3))
```

Use the measured t_search from around the query loop. One second is one million microseconds.

Q21. Measure successful and unsuccessful searches separately

```
hits = random.choices(vocab, k=50000)
misses = ["__definitely_missing_" + str(i)
          for i in range(50000)]

start = time.perf_counter()
for word in hits:
    m.search(index, word)
hit_time = time.perf_counter() - start

start = time.perf_counter()
for word in misses:
    m.search(index, word)
miss_time = time.perf_counter() - start

print(name, "hit:", round(hit_time, 4),
      "miss:", round(miss_time, 4))
```

Use time.perf_counter() for elapsed-time measurement. For guaranteed misses, verify each generated miss is absent from the vocabulary before timing.

Q22. Add a correctness check before timing

```
for word in vocab[:100]:
    if m.search(index, word) is None:
        raise AssertionError("Missing indexed word: " + word)
```

Checks a sample of known words. Run correctness checks outside the timed region so they do not contaminate benchmark measurements.

Q23. Save benchmark timings to CSV

```
import csv

with open("benchmark_results.csv", "w", newline="",
          encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["distinct_words", "structure",
                     "build_seconds", "search_seconds",
                     "print_seconds"])
    for name in modules:
        for i, size in enumerate(SIZES):
            writer.writerow([
                size, name,
                times[name]["build"][i],
                times[name]["search"][i],
                times[name]["print"][i]
            ])
```

Run after all benchmark loops. This assumes each timing list contains one value for every entry in SIZES.

Part F — Common viva / explanation questions

• Why does dictionary output use sorted(index)? Because insertion order is not necessarily lexicographic.

• Why is LineQueue not a frequency counter? It stores line numbers; the same word may occur multiple times on one line.

• Why does Trie search depend on query length? It follows one child edge per character.

• What does BST in-order traversal guarantee? Ascending lexicographic order for the BST's comparison rule.

• Why should benchmark setup be outside the timer? To measure the intended operation rather than data generation or unrelated work.

• What are the three benchmark actions? Build the index, search 100,000 random words, and write sorted output.

Last-minute priority list

- Command-line arguments and safe file handling
- get_words() and case normalization
- LineQueue.add(), count(), and frequency versus line numbers
- Sorted output for all three data structures
- Trie prefix search and BST in-order traversal
- Benchmark timing, reproducibility and correctness checks

Study tip: Practise each change in the supplied code files, not just as an isolated snippet. Several modifications require updating more than one function.